

7. Формы: React Hook Form и Tailwind CSS

Цель занятия

На прошлых занятиях мы изучили компоненты, состояние, props, списки и условный рендеринг. Теперь переходим к формам. Формы нужны почти в любом приложении: регистрация, обратная связь, поиск, фильтры, настройки профиля.

В этой методичке мы используем:

- `input`, `select`, `textarea`;
- React Hook Form для работы с данными формы;
- Tailwind CSS для быстрой стилизации;
- `register`, `handleSubmit`, `formState.errors`, `reset`, `useWatch`.

После занятия должны уметь:

- объяснить, зачем нужны формы;
- подключить Tailwind CSS к Vite-проекту;
- подключить React Hook Form;
- зарегистрировать поле через `register`;
- обработать отправку формы через `handleSubmit`;
- показать ошибки валидации;
- сбросить форму через `reset`;
- читать значение поля через `useWatch`;
- использовать utility-классы Tailwind CSS в `className`.

1. Что говорит документация

React Hook Form предлагает хук `useForm`, который возвращает методы для регистрации полей, обработки отправки и чтения состояния формы.

Базовая схема:

```
javascript
const {
  register,
  handleSubmit,
  formState: { errors },
} = useForm()
```

Tailwind CSS предлагает писать стили через utility-классы прямо в `className`.

Пример:

```
javascript

```

Вместо отдельного CSS-класса для каждого поля мы собираем внешний вид из готовых маленьких классов.

2. Почему используем React Hook Form

Форму можно сделать на обычном `useState`, но с ростом количества полей код быстро становится длинным.

React Hook Form помогает:

- регистрировать поля;
- собирать данные формы;
- валидировать значения;
- показывать ошибки;
- сбрасывать форму;
- меньше писать однотипный код.

Пример:

```
javascript
const { register, handleSubmit } = useForm()
```

Поле:

```
javascript

```

Отправка:

```
javascript
<form onSubmit={handleSubmit(onSubmit)}>
```

3. Почему используем Tailwind CSS

Tailwind CSS - это utility-first CSS-фреймворк. Он позволяет описывать внешний вид прямо в `className`.

Обычный CSS-подход:

```
css
.field {
  border-radius: 6px;
  padding: 8px 12px;
}
```

Tailwind-подход:

```
javascript

```

Плюсы для учебного проекта:

- стили видны рядом с JSX;
- легко быстро собрать форму;
- меньше отдельных CSS-классов;
- удобно объяснять отступы, рамки, сетку и состояния фокуса.

4. Установка зависимостей

Для этого этапа были установлены пакеты:

```
bash
npm install react-hook-form tailwindcss @tailwindcss/vite
```

В `package.json` появились зависимости:

```
json
{
  "dependencies": {
    "@tailwindcss/vite": "^4.3.0",
    "react-hook-form": "^7.77.0",
    "tailwindcss": "^4.3.0"
  }
}
```

Версии могут отличаться, если проект устанавливается позже. Важно понимать назначение пакетов:

- `react-hook-form` - работа с формами;
- `tailwindcss` - utility-классы;
- `@tailwindcss/vite` - подключение Tailwind к Vite.

5. Подключение Tailwind к Vite

В `vite.config.js` добавляется плагин:

```
javascript
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

Теперь Vite умеет обрабатывать Tailwind-классы.

6. Подключение Tailwind в CSS

В `src/index.css` добавляется:

```
css
@import "tailwindcss";
```

После этого в JSX можно использовать Tailwind-классы:

```
javascript
<div className="grid gap-4">
  ...
</div>
```

7. Импорт React Hook Form

В `App.jsx` добавляем:

```
javascript
import { useForm, useWatch } from "react-hook-form";
```

`useForm` нужен для создания формы.

`useWatch` нужен, чтобы следить за значением одного поля. В нашем примере мы смотрим, какая тема выбрана в `select`.

8. Создание формы

Создадим компонент:

```
javascript
function ContactForm() {
  const [submittedContact, setSubmittedContact] = useState(null)

  const {
    register,
    handleSubmit,
    control,
    reset,
    formState: { errors, isSubmitting },
  } = useForm({
    defaultValues: {
      name: '',
      email: '',
      topic: 'React',
      message: '',
    },
  })

  const selectedTopic = useWatch({
    control,
    name: 'topic',
  })

  function onSubmit(data) {
    setSubmittedContact(data)
    reset()
  }

  return <form onSubmit={handleSubmit(onSubmit)}>{/* поля */}</form>
}
```

Что здесь происходит:

- `submittedContact` хранит данные последней отправки;
- `register` подключает поля к React Hook Form;
- `handleSubmit` обрабатывает отправку;
- `control` нужен для `useWatch`;
- `reset` очищает форму;
- `errors` хранит ошибки;
- `isSubmitting` показывает состояние отправки;
- `defaultValues` задает начальные значения.

9. Регистрация поля через `register`

Поле имени:

```
javascript
<input
  type="text"
  {...register("name", {
    required: "Введите имя",
    minLength: {
      value: 2,
      message: "Минимум 2 символа",
    },
  })}
/>
```

`register("name")` говорит React Hook Form: это поле называется `name`.

Правила:

```
text
  required -> поле обязательно
  minLength -> минимальная длина
```

Если пользователь нарушит правило, ошибка появится в `errors.name`.

10. Показ ошибки

```
javascript
{errors.name && (
  <span className="text-sm text-red-600">
    {errors.name.message}
  </span>
)}
```

Как читается:

```
text
  Если есть ошибка errors.name, показать текст ошибки.
```

Это условный рендеринг из прошлой методички.

11. Email с pattern

Для email можно добавить простую проверку:

```
javascript
<input
  type="email"
  {...register("email", {
    required: "Введите email",
    pattern: {
      value: /\S+@\S+\.\S+/,
      message: "Введите корректный email",
    },
  })}
/>
```

`pattern` проверяет значение по регулярному выражению.

12. Select и `useWatch`

Поле выбора темы:

```
javascript
<select {...register("topic")}>
  <option value="React">React</option>
  <option value="Vite">Vite</option>
  <option value="Tailwind CSS">Tailwind CSS</option>
  <option value="React Hook Form">React Hook Form</option>
</select>
```

Чтобы показать выбранную тему прямо в интерфейсе:

```
javascript
const selectedTopic = useWatch({
  control,
  name: "topic",
})
```

И в JSX:

```
javascript
<p>
  Сейчас выбрана тема: <strong>{selectedTopic}</strong>
</p>
```

Когда пользователь меняет `select`, текст рядом тоже обновляется.

13. Textarea

Поле сообщения:

```
javascript
<textarea
  {...register("message", {
    required: "Введите сообщение",
    minLength: {
      value: 10,
      message: "Минимум 10 символов",
    },
  })}
/>
```

`textarea` используется для длинного текста.

14. Отправка формы

Форма:

```
javascript
<form onSubmit={handleSubmit(onSubmit)}>
```

Функция:

```
javascript
function onSubmit(data) {
  setSubmittedContact(data)
  reset()
}
```

`data` - объект со значениями полей.

Пример:

```
javascript
{
  name: 'Анна',
  email: 'anna@example.com',
  topic: 'React',
  message: 'Хочу узнать больше про формы'
}
```

15. Кнопка отправки

```
javascript
<button type="submit" disabled={isSubmitting}>
  Send
</button>
```

`type="submit"` отправляет форму.

`disabled={isSubmitting}` блокирует кнопку во время отправки.

16. Кнопка сброса

```
javascript
<button type="button" onClick={() => reset()}>
  Clear
</button>
```

Важно: для обычной кнопки внутри формы пишем:

```
javascript
type="button"
```

Иначе браузер может воспринимать кнопку как отправку формы.

17. Tailwind-классы на форме

Пример:

```
javascript
<form className="grid gap-4">
```

Здесь:

- `grid` включает CSS Grid;
- `gap-4` задает расстояние между элементами.

Пример поля:

```
javascript
<input className="rounded-md border px-3 py-2 outline-none" />
```

Здесь:

- `rounded-md` скругляет углы;
- `border` добавляет рамку;
- `px-3` задает горизонтальные отступы;
- `py-2` задает вертикальные отступы;
- `outline-none` убирает стандартную обводку.

18. Tailwind и CSS-переменные проекта

В проекте уже есть CSS-переменные:

```
css
--text
--text-h
--bg
--border
--accent
```

Их можно использовать внутри Tailwind-классов:

```
javascript
className="border-[var(--border)] text-[var(--text-h)]"
```

Так форма остается в визуальном стиле проекта.

19. Частые ошибки

Ошибка 1. Забыть `handleSubmit`

Неправильно:

```
javascript
<form onSubmit={onSubmit}>
```

Правильно:

```
javascript
<form onSubmit={handleSubmit(onSubmit)}>
```

Ошибка 2. Забыть spread перед `register`

Неправильно:

```
javascript
<input register("name") />
```

Правильно:

```
javascript
<input {...register("name")} />
```

Ошибка 3. Писать обычную кнопку без `type="button"`

Неправильно:

```
javascript
<button onClick={() => reset()}>Clear</button>
```

Правильно:

```
javascript
<button type="button" onClick={() => reset()}>
  Clear
</button>
```

Ошибка 4. Искать ошибку не в том поле

Если поле зарегистрировано как:

```
javascript
  register("email")
```

ошибка будет в:

```
javascript
  errors.email
```

А не в:

```
javascript
  errors.mail
```

20. Практика

Задание 1

Проверьте, что Tailwind подключен:

```
css
@import "tailwindcss";
```

Задание 2

Создайте компонент `ContactForm` и подключите:

```
javascript
const {
  register,
  handleSubmit,
  reset,
  formState: { errors },
} = useForm()
```

Задание 3

Добавьте поле имени с валидацией:

```
javascript
<input
  type="text"
  {...register("name", {
    required: "Введите имя",
    minLength: {
      value: 2,
      message: "Минимум 2 символа",
    },
  })}
/>
```

Задание 4

Добавьте вывод ошибки:

```
javascript
{errors.name && <span>{errors.name.message}</span>}
```

Задание 5

Добавьте отправку формы:

```
javascript
function onSubmit(data) {
  console.log(data)
}

<form onSubmit={handleSubmit(onSubmit)}>
```

21. Контрольные вопросы

1. Для чего нужны формы?
2. Что делает React Hook Form?
3. Что делает useForm?
4. Что делает register?
5. Почему перед register пишется {...}?
6. Что делает handleSubmit?
7. Где хранятся ошибки формы?
8. Что делает reset?
9. Для чего нужен useWatch?
10. Что значит type="submit"?
11. Почему для кнопки сброса нужен type="button"?
12. Что такое Tailwind CSS?
13. Что делает класс grid?
14. Что делает класс gap-4?
15. Как использовать CSS-переменную внутри Tailwind-класса?

22. Краткий итог

React Hook Form помогает работать с формами без большого количества ручного состояния. Tailwind CSS помогает быстро стилизовать форму через utility-классы.

Главная схема:

```
text
register -> подключает поле
handleSubmit -> обрабатывает отправку
errors -> хранит ошибки
reset -> очищает форму
useWatch -> следит за значением поля
```